Building and Analyzing Monolingual Transformer LMs in Two Indian Languages

Phase 1 — Data Collection and Tokenizer Construction [25 Marks]

> **Language Models and Agents — Monsoon 2026, Individual Project Author:** Satyajit Priyadarshi (2025201055) **Deadline:** 19th August 2026, 11:59 P.M.

Dataset Paths

- Hindi dataset on Kaggle
- Nepali dataset on Kaggle

Table of Contents

1. Overview & Language Selection
2. Data Sources
3. Data Collection Pipeline
4. Preprocessing Pipeline
5. Corpus Statistics
6. Train / Validation / Test Splits
7. Tokenizer Training Methodology
8. Tokenizer Results
9. Deliverables Mapping
10. Repository Structure & How to Reproduce
11. Limitations, Risks & Future Work
12. Appendix: Script Reference

1. Overview & Language Selection

This phase builds two **independent** data + tokenizer pipelines, one per language tier, with no document sharing, no concatenation, and no shared vocabulary between the two.

	Model H (higher-resource)	Model L (lower-resource)
Language	**Hindi**	**Nepali**
Script	Devanagari	Devanagari
Target corpus size	~500M tokens	~500M tokens (from allowed list)

1.1 Model H — Hindi (higher-resource)

Hindi was chosen as the higher-resource language because it has by far the largest volume of freely available, high-quality Devanagari-script web text of any Indian language: large curated public corpora (AI4Bharat Sangraha, IndicCorp, CC-100, OSCAR), dozens of major national newspapers and broadcasters publishing daily editorial content online, large digitized literature archives (Kavita Kosh, Gadya Kosh, Rekhta, Hindi Samay), and a mature Wikipedia/Wikisource. This made it realistic to reach the ~500M token target using a mix of self-collected crawl data and a public corpus, while sharing a script (Devanagari) with the chosen lower-resource language — useful for later cross-lingual comparison of tokenizer fertility and model behaviour.

1.2 Model L — Nepali (lower-resource)

Nepali was selected from the required list (Assamese, Bhojpuri, Bodo, Dogri, Konkani, Maithili, Manipuri, Mizo, Nepali, Sindhi) because, relative to the other options, it has:

- A genuinely active Nepali-language news ecosystem (Ekantipur, Onlinekhabar, Setopati, Ratopati, Nagarik News, and others) publishing original editorial content daily, giving a strong manual web-crawl source;
- Existing coverage in AI4Bharat Sangraha (`verified/nep`) as a public corpus to reach scale;
- Devanagari script, which let the same crawler architecture, cleaning heuristics (Devanagari ratio filtering, NFC normalization) and tokenizer training code be reused/adapted for both languages with only light, language-specific tuning — keeping the two pipelines independent but comparable.

No lower-resource-language token shortfall was encountered against the manual-collection component (~99M tokens crawled per language, see §5); the overall ~500M target is reached by combining this with the public Sangraha component (§3.2).

---

2. Data Sources

The assignment requires at least **5 sources** per language and **≥20% of final training tokens from manual collection**. Manual collection here means an original, checkpointed, robots.txt- respecting web crawler (`hindi_web_crawler.py` / `nepali_web_crawler.py`) written for this project, targeting **human-authored editorial content only** (no aggregator/MT pages) — as opposed to the already-cleaned public corpus.

2.1 Hindi sources

#	Source category	Examples in seed list
1	National news portals	BBC Hindi, DW Hindi, Jagran, Navbharat Times, Amar Ujala, Patrika, Aaj Tak, NDTV, Zee News, News18 Hindi, ABP Live, India TV, The Hindu (Hindi), Hindustan Times (Hindi), Punjab Kesari, Live Hindustan
2	Regional/wire dailies	Deshbandhu, Rajasthan Patrika, One India Hindi, Asianet News Hindi
3	Magazines / long-form editorial	Outlook Hindi, Dainik Bhaskar, Bhaskar, Pratilipi
4	Literature / long-form archives	Kavita Kosh, Gadya Kosh, Sahitya Kunj, Hindi Samay, Rekhta
5	Encyclopedic (human-edited)	Hindi Wikipedia, Hindi Wikisource
6	**Public corpus**	AI4Bharat **Sangraha** (`verified/hin` split), streamed via HuggingFace `datasets`

2.2 Nepali sources

#	Source category	Examples in seed list
1	National news portals	Ekantipur, Onlinekhabar, Setopati, Nagarik News, Ratopati
2	International broadcasters (Nepali edition)	BBC Nepali, DW Nepali
3	Regional dailies / news sites	Naya Patrika Daily, Annapurna Post, Nepal Press, Himal Khabar, Nepal Khabar, Baahrakhari, Thaha Khabar, Karobar Daily, Rajdhani Daily, DC Nepal, Ujyaalo Online, Nepal News
4	Literature	Nepali Sahitya
5	Encyclopedic (human-edited)	Nepali Wikipedia, Nepali Wikisource
6	**Public corpus**	AI4Bharat **Sangraha** (`verified/nep` split), downloaded directly from the HF Hub as Parquet shards

Both languages therefore draw on 5+ independent editorial/manual source categories, plus one public corpus, satisfying the source-diversity requirement while keeping each language's corpus fully separate from the other's.

---

3. Data Collection Pipeline

3.1 Custom web crawler (manual-collection component)

`hindi_web_crawler.py` and `nepali_web_crawler.py` are original, asyncio-based crawlers built for this project (not a third-party scraping tool). Key design points, identical in spirit across both languages:

- **Politeness & legality**: robots.txt is fetched and cached per domain (`get_robots` / `robots_allows`); a descriptive academic `User-Agent` string is sent; requests to a given domain are rate-limited (`PER_DOMAIN_DELAY = 1.0s`) via `polite_wait` .
- **Breadth-first frontier crawl**: seeded from a curated list of editorial domains (`SEED_URLS` → `ALLOWED_DOMAINS`), extracting article text with `trafilatura` and discovering new in-domain links with `BeautifulSoup` .
- **Quality filtering at fetch time**: pages are kept only if their Devanagari-character ratio clears a threshold (`MIN_DEVANAGARI_RATIO` , 0.55) and they exceed a minimum word count (`MIN_DOC_WORDS` , 80), to exclude non-Hindi/Nepali or boilerplate-only pages before they ever reach the corpus.
- **Text normalization at capture time**: Unicode NFC normalization and whitespace collapsing (`normalize_text`).
- **Deduplication in-flight**: a cheap document signature (first 200 + last 200 characters + a length bucket, SHA-1 hashed) is checked against `seen_hashes` before a page is accepted, to avoid storing the same article twice within a single crawl.
- **Checkpoint/resume**: crawler state (visited URLs, seen hashes, frontier, running token/doc counts, shard index) is written to `checkpoint.json` every 100 documents, so a run can be safely killed and resumed with the same command — this is how the Nepali crawl was extended across two separate runs (§5.1).
- **Sharded output**: cleaned documents are appended as JSON Lines to `shard_NNNNNN.jsonl` files (2,000 docs/shard), each record holding the URL, extracted text, and an estimated token count.
- **Stop conditions**: a run stops when it hits a token budget (`TARGET_TOKENS`) or a wallclock safety limit (`MAX_WALLCLOCK_HOURS`), whichever comes first.
- **Token estimation during crawling**: `estimate_tokens()` uses a fixed proxy of **1.7 tokens/word** for Devanagari text (a standard rough multiplier for BPE over this script), used consistently through crawling, downloading, and the final corpus-statistics report. Final, exact token counts are computed later from the actual trained tokenizer (§8).

Two Nepali crawler script versions are included: `nepali_web_crawler_v1_69M.py` (the original run, `TARGET_TOKENS = 100_000_000` , executed to its 7.5h wallclock cap) and `nepali_web_crawler.py` (a second, resumed/continuation run with `TARGET_TOKENS` lowered to top up an additional ~40M tokens). `Proof_data_collection_log.log` contains the unedited console output of all three crawl runs (two Nepali, one Hindi) as evidence of execution.

3.2 Public corpus download

- **Hindi** — `download_and_merge.py` streams the AI4Bharat **Sangraha** dataset (`ai4bharat/sangraha` , `verified/hin` split) via the HuggingFace `datasets` library in streaming mode, writing one flattened document per line until an estimated 400,000,000-token budget is reached (same 1.7 tokens/word proxy as the crawler).
- **Nepali** — `download_merge_nepali.py` downloads the `verified/nep` Parquet shards directly from the HF Hub (`huggingface_hub.hf_hub_download`) and streams them with `pyarrow` , avoiding loading the full `datasets` object into memory; each downloaded Parquet file is deleted immediately after processing to bound disk usage. It targets the same 400M-token budget.

Both scripts then call `merge_with_dedup()` , which concatenates the cleaned crawl file and the downloaded public file and drops any line that is an **exact SHA-256 duplicate** of a line already seen — catching cases where the same news article was picked up both by the custom crawler and by Sangraha's own web crawl.

---

4. Preprocessing Pipeline

`preprocess_hindi_corpous.py` and `preprocess_nepali_corpus.py` turn the raw `shard_*.jsonl` crawl output into a single clean plaintext file (one document per line), ready to merge with the public corpus and feed into tokenizer training. Both scripts share a common pipeline, with a couple of Nepali-specific additions:

Shared cleaning steps (both languages):

1. Load every `shard_*.jsonl` file in the crawl output directory.
2. Unicode **NFC normalization** of each document's raw text.
3. Strip known **site boilerplate blocks** via hand-curated regexes (cookie-consent banners, app promo footers, etc.).
4. **Line-level language filtering**: drop any line under `MIN_LINE_CHARS` (8) characters, drop lines matching known **navigation/noise patterns** ("यह भी पढ़ें" / "Also Read" / etc.), and drop any remaining line whose Devanagari-to-(Devanagari+Latin) character ratio is below `MIN_LINE_DEVANAGARI_RATIO` (0.5) — this removes embedded English UI strings and captions.
5. Flatten the surviving lines of a document into a single line (tokenizer training corpora conventionally want one document/sentence per line, no embedded newlines).

6. **Whole-document drop** if the raw text contains any marker of a template/utility page (weather widgets, subscription FAQs, IPO listing tables, homepage headline dumps) — these are fluent Hindi/Nepali but not articles, and are near-duplicated across thousands of URLs.
7. Drop documents that fall below `MIN_DOC_WORDS_AFTER_CLEAN` (40 words) after cleaning — mostly pages that were almost entirely boilerplate or partly in English.
8. **Deduplicate**: an exact-hash check (SHA-256 of the cleaned text) plus a near-duplicate check re-using the crawler's own signature (first 200 + last 200 chars + length band), to catch the same article picked up twice across separate/resumed crawl runs.
9. Write one cleaned document per line to the output file, and print a full accounting: docs in → docs out, words in → words out, and a breakdown of why each dropped document was dropped.

Nepali-specific additions (`preprocess_nepali_corpus.py`):

- **(A) Index/homepage-page detection by URL structure**: a document is dropped outright if its URL's last path segment contains neither a digit nor a hyphen — real articles in this crawl are always served at URLs like `/news/405822` or `/agriculture-officials-inspect-...` , whereas homepage/category root URLs (e.g. `karobardaily.com/`) have neither and are headline aggregations rather than articles.
- **(B) Within-document exact-line dedup**: some sites (e.g. Ekantipur) repeat the same headline paragraph twice on one page (sidebar + main list); duplicate lines within a single document are dropped before flattening.

---

5. Corpus Statistics

5.1 Manual web-crawl yield (evidence: `Proof_data_collection_log.log`)

Nepali — two checkpointed runs (run 2 resumed the crawl to top up beyond run 1's stop point):

Run	Wallclock	Documents	Est. tokens	Stop reason
Run 1 (`nepali_web_crawler_v1_69M.py`)	7.50 h	59,372	59,033,300	token target reached
Run 2 (`nepali_web_crawler.py`)	4.98 h	40,524	40,036,604	token target reached
Total	**12.48 h**	**99,896**	**~99,069,904**	—

Hindi — single run (`hindi_web_crawler.py`):

Run	Wallclock	Documents	Est. tokens	Stop reason
Run 1	8.92 h	93,784	98,820,416	wallclock safety limit reached

All figures above are taken directly from the crawler's own console log (unedited), not recomputed after the fact — they use the crawler's 1.7-tokens/word proxy, consistent with how the 400M-token public-corpus budget was also estimated.

5.2 Public corpus (AI4Bharat Sangraha)

Language	Split	Budget targeted	Download method
Hindi	`verified/hin`	400,000,000 tokens (est.)	HuggingFace `datasets` , streaming
Nepali	`verified/nep`	400,000,000 tokens (est.)	Direct Parquet download via `huggingface_hub` + `pyarrow`

5.3 Final merged corpus and manual-vs-downloaded split

The final corpus for each language is `merge_with_dedup(public_file, crawl_clean_file)` : the two sources concatenated and deduplicated on exact SHA-256 line match. Combining the manual-crawl totals above with the targeted 400M-token public budget:

Language	Manual (crawl) tokens	Public (Sangraha) tokens (budget)	Combined total (pre-dedup)	Manual fraction
Nepali	~99.07M	~400.00M	~499.07M	**~19.85%**
Hindi	~98.82M	~400.00M	~498.82M	**~19.81%**

---

6. Train / Validation / Test Splits

Implemented identically for both languages inside the tokenizer training scripts (`split_train_val_test`), operating on the final merged corpus, one document per line:

- A single seeded RNG (`SEED = 42`) makes a **document-level**, **line-independent** random assignment for every line of the corpus — so the split is reproducible and does not depend on file order.
- **Train** (~98%): fits the SentencePiece BPE tokenizer (the merge table is learned from this data only).
- **Validation** (`VAL_FRACTION = 0.01` , ~1%): used **only** to choose the vocabulary size — never used to fit merges, never used for the final reported numbers.
- **Test** (`TEST_FRACTION = 0.01` , ~1%): held out completely, touched exactly once, after the vocabulary size has already been chosen from validation — used only to produce the final, unbiased reported statistics .
- Splits are cached to disk (`tokenizer_work_{lang}/{lang}_{train,val,test}.txt`) so re-running the training script doesn't reshuffle the data.

This gives per-language, non-overlapping train/validation/test files, computed independently for Hindi and Nepali (no shared documents, no shared splits).

---

7. Tokenizer Training Methodology

`train_tokenizer_hindi_final_3_split.py` and `train_tokenizer_nepali_final_3_split.py` train a **SentencePiece BPE** tokenizer from scratch, independently, per language — no pretrained tokenizer or shared vocabulary is used anywhere in this pipeline.

1. **Candidate sweep**: four vocabulary sizes are trained from the train split — `[16000, 32000, 50000, 64000]` — each a full from-scratch SentencePiece BPE run (`character_coverage = 0.9995` , `unk_id/bos_id/eos_id/pad_id` fixed at 0–3).
2. **Selection metric**: each candidate is evaluated on the held-out validation split, computing **fertility** (tokens ÷ whitespace-delimited words), **UNK rate**, and **average characters per token**.
3. **Selection rule**: the smallest vocabulary size whose validation fertility is within a 3% tolerance of the best fertility achieved by any candidate is chosen — i.e. we prefer a smaller, cheaper vocabulary unless a larger one gives a meaningfully better (lower) fertility. This keeps the choice principled rather than "biggest vocab wins" or eyeballed.
4. **Final, unbiased report**: only after the vocabulary size is locked in does the chosen model get evaluated on the untouched test split — this produces the numbers reported in §8 (token-frequency table, fertility, UNK rate, avg chars/token, tokenization examples). The test set never influences which vocabulary size was picked.
5. The winning model/vocab files are copied to `final/{lang}_tokenizer.model` and `final/{lang}_tokenizer.vocab` — these are deliverables #6 and #7.

---

8. Tokenizer Results

8.1 Nepali tokenizer

Validation sweep (used only to select vocab size):

Vocab size	Fertility	UNK rate	Chars/token
16,000	1.415	0.00410	4.56
32,000	1.298	0.00447	4.97
50,000 chosen	**1.246**	0.00466	5.18
64,000	1.224	0.00474	5.28

Chosen `vocab_size = 50,000` — although 64k has slightly lower validation fertility (1.224 vs 1.246), the 50k model is within the 3% tolerance and gives the better size/performance trade-off. The examples below support this choice: at 50k, long Nepali words become substantially more complete subword pieces than at 16k, while the piece counts are already identical to 64k for the first example and identical or nearly identical for the other examples.

Final report, computed on the held-out TEST split:

Metric	Value
Fertility	1.242
Avg. chars/token	5.20
Unique tokens observed	49,748
Total tokens	3,399,126
UNK tokens	14,665 (0.431%)

Tokenization examples:

Example 1 — अन्तरमहाद्वीपीय वैज्ञानिक अनुसन्धानमा बहुआयामिकरण र विद्युतीकरणको महत्वपूर्ण भूमिका छ।

Vocabulary size	Tokens	Number of pieces
16,000	`['_अन्तर', 'महा', 'द्', 'वी', 'प', 'ीय', '_वैज्ञानिक', '_अनुसन्धानमा', '_बहु', 'आ', 'या', 'मिक', 'रण', '_र', '_विद्यु', 'ती', 'करणको', '_महत्वपूर्ण', '_भूमिका', '_छ', '।']`	21
32,000	`['_अन्तर', 'महा', 'द्वी', 'प', 'ीय', '_वैज्ञानिक', '_अनुसन्धानमा', '_बहुआयामिक', 'रण', '_र', '_विद्युती', 'करणको', '_महत्वपूर्ण', '_भूमिका', '_छ', '।']`	16
50,000	`['_अन्तर', 'महा', 'द्वी', 'प', 'ीय', '_वैज्ञानिक', '_अनुसन्धानमा', '_बहुआयामिक', 'रण', '_र', '_विद्युती', 'करणको', '_महत्वपूर्ण', '_भूमिका', '_छ', '।']`	**16**
64,000	`['_अन्तर', 'महा', 'द्वी', 'प', 'ीय', '_वैज्ञानिक', '_अनुसन्धानमा', '_बहुआयामिक', 'रण', '_र', '_विद्युती', 'करणको', '_महत्वपूर्ण', '_भूमिका', '_छ', '।']`	16

Example 2 — अन्तरसरकारी जलवायुपरिवर्तनसम्बन्धी अनुसन्धान संस्थानले जैवविविधतासंरक्षण, वातावरणीय पुनर्संरचनात्मक योजनाहरू र दिगो विकासकार्यक्रमको प्रभावकारिताबारे विस्तृत प्रतिवेदन सार्वजनिक गऱ्यो।

Vocabulary size	Tokens	Number of pieces
16,000	`['_अन्तर', 'सरकारी', '_जलवायु', 'परि', 'वर्त', 'न', 'सम्बन्धी', '_अनुसन्धान', '_संस्थान', 'ले', '_जै', 'व', 'वि', 'विध', 'ता', 'संर', 'क्षण', ',', '_वातावरणीय', '_पुनर्', 'संरचना', 'त्मक', '_योजना', 'हरू', '_र', '_दिगो', '_विकास', 'कार्य', 'क्रम', 'को', '_प्रभाव', 'कारिता', 'बारे', '_विस्तृत', '_प्रतिवेदन', '_सार्वजनिक', '_गर्', '\u200d', 'यो', '।']`	40
32,000	`['_अन्तर', 'सरकारी', '_जलवायु', 'परि', 'वर्तन', 'सम्बन्धी', '_अनुसन्धान', '_संस्थानले', '_जै', 'व', 'वि', 'विध', 'ता', 'संरक्षण', ',', '_वातावरणीय', '_पुनर्संरचना', 'त्मक', '_योजनाहरू', '_र', '_दिगो', '_विकास', 'कार्य', 'क्रम', 'को', '_प्रभावकारिता', 'बारे', '_विस्तृत', '_प्रतिवेदन', '_सार्वजनिक', '_गर्', '\u200d', 'यो', '।']`	34
50,000	`['_अन्तर', 'सरकारी', '_जलवायु', 'परिवर्तन', 'सम्बन्धी', '_अनुसन्धान', '_संस्थानले', '_जै', 'व', 'वि', 'विध', 'ता', 'संरक्षण', ',', '_वातावरणीय', '_पुनर्संरचना', 'त्मक', '_योजनाहरू', '_र', '_दिगो', '_विकास', 'कार्यक्रम', 'को', '_प्रभावकारिता', 'बारे', '_विस्तृत', '_प्रतिवेदन', '_सार्वजनिक', '_गर्', '\u200d', 'यो', '।']`	**32**
64,000	`['_अन्तर', 'सरकारी', '_जलवायु', 'परिवर्तन', 'सम्बन्धी', '_अनुसन्धान', '_संस्थानले', '_जै', 'व', 'वि', 'विध', 'ता', 'संरक्षण', ',', '_वातावरणीय', '_पुनर्संरचना', 'त्मक', '_योजनाहरू', '_र', '_दिगो', '_विकास', 'कार्यक्रम', 'को', '_प्रभावकारिता', 'बारे', '_विस्तृत', '_प्रतिवेदन', '_सार्वजनिक', '_गर्', '\u200d', 'यो', '।']`	32

Example 3 — संविधानसभाको पुनर्संरचनासम्बन्धी अन्तर्राष्ट्रिय सहकार्यले लोकतान्त्रिक संस्थागत विकासलाई सुदृढ बनायो।

Vocabulary size	Tokens	Number of pieces
16,000	`['_संविधानसभा', 'को', '_पुनर्', 'संरचना', 'सम्बन्धी', '_अन्तर्राष्ट्रिय', '_सहकार्य', 'ले', '_लोकतान्त्रिक', '_संस्थागत', '_विकास', 'लाई', '_सुदृढ', '_बनायो', '।']`	15
32,000	`['_संविधानसभाको', '_पुनर्संरचना', 'सम्बन्धी', '_अन्तर्राष्ट्रिय', '_सहकार्यले', '_लोकतान्त्रिक', '_संस्थागत', '_विकासलाई', '_सुदृढ', '_बनायो', '।']`	11
50,000	`['_संविधानसभाको', '_पुनर्संरचना', 'सम्बन्धी', '_अन्तर्राष्ट्रिय', '_सहकार्यले', '_लोकतान्त्रिक', '_संस्थागत', '_विकासलाई', '_सुदृढ', '_बनायो', '।']`	**11**
64,000	`['_संविधानसभाको', '_पुनर्संरचना', 'सम्बन्धी', '_अन्तर्राष्ट्रिय', '_सहकार्यले', '_लोकतान्त्रिक', '_संस्थागत', '_विकासलाई', '_सुदृढ', '_बनायो', '।']`	11

Top tokens (TEST set) are dominated by high-frequency function words and punctuation — sentence terminators (`_।` , `।`), the conjunction `_र` ("and"), the copula `_छ` / `_छन्` / `_हो` , and common verbal/postposition suffixes (`_भएको` , `_लागि` , `_गर्न` , `_गरेको`) — consistent with a morphologically productive BPE vocabulary that has learned frequent Nepali grammatical morphemes as single pieces rather than falling back to characters.

8.2 Hindi tokenizer

Validation sweep:

Vocab size	Fertility	UNK rate	Chars/token
16,000	1.329	0.00198	3.84
32,000	1.249	0.00211	4.08
50,000 chosen	**1.215**	0.00217	4.20
64,000	1.200	0.00220	4.25

Chosen `vocab_size = 50,000` — best achievable validation fertility was 1.200 (at 64k); 50k's 1.215 is within tolerance. The 50k model is therefore selected because it gives a strong fertility result with fewer entries than 64k. On the example sentences below, 50k already produces the same compact segmentation as 64k, while 16k and 32k split the longer words into more pieces.

Final report, computed on the held-out TEST split:

Metric	Value
Fertility	1.215
Avg. chars/token	4.20
Unique tokens observed	47,714
Total tokens	3,376,802
UNK tokens	8,553 (0.253%)

Tokenization examples:

Example 1 — भारत की अर्थव्यवस्था में प्रौद्योगिकी और अंतरराष्ट्रीय सहयोग महत्वपूर्ण भूमिका निभाते हैं।

Vocabulary size	Tokens	Number of pieces
16,000	`['_भारत', '_की', '_अर्थव्यवस्था', '_में', '_प्रौद्योगिकी', '_और', '_अंतरराष्ट्रीय', '_सहयोग', '_महत्वपूर्ण', '_भूमिका', '_निभाते', '_हैं', '।']`	13
32,000	`['_भारत', '_की', '_अर्थव्यवस्था', '_में', '_प्रौद्योगिकी', '_और', '_अंतरराष्ट्रीय', '_सहयोग', '_महत्वपूर्ण', '_भूमिका', '_निभाते', '_हैं', '।']`	13
50,000	`['_भारत', '_की', '_अर्थव्यवस्था', '_में', '_प्रौद्योगिकी', '_और', '_अंतरराष्ट्रीय', '_सहयोग', '_महत्वपूर्ण', '_भूमिका', '_निभाते', '_हैं', '।']`	**13**
64,000	`['_भारत', '_की', '_अर्थव्यवस्था', '_में', '_प्रौद्योगिकी', '_और', '_अंतरराष्ट्रीय', '_सहयोग', '_महत्वपूर्ण', '_भूमिका', '_निभाते', '_हैं', '।']`	13

Example 2 — अंतरमहाद्वीपीय वैज्ञानिक अनुसंधान में बहुआयामीकरण और विद्युतीकरण की महत्वपूर्ण भूमिका है।

Vocabulary size	Tokens	Number of pieces
16,000	`['_अंतर', 'महा', 'द्', 'वी', 'पीय', '_वैज्ञानिक', '_अनुसंधान', '_में', '_बह', 'ुआ', 'या', 'मी', 'करण', '_और', '_विद्य', 'ुत', 'ीकरण', '_की', '_महत्वपूर्ण', '_भूमिका', '_है', '।']`	23
32,000	`['_अंतर', 'महा', 'द्वी', 'पीय', '_वैज्ञानिक', '_अनुसंधान', '_में', '_बह', 'ुआ', 'या', 'मी', 'करण', '_और', '_विद्यु', 'तीकरण', '_की', '_महत्वपूर्ण', '_भूमिका', '_है', '।']`	20
50,000	`['_अंतर', 'महा', 'द्वीपीय', '_वैज्ञानिक', '_अनुसंधान', '_में', '_बहुआयामी', 'करण', '_और', '_विद्युतीकरण', '_की', '_महत्वपूर्ण', '_भूमिका', '_है', '।']`	**15**
64,000	`['_अंतर', 'महा', 'द्वीपीय', '_वैज्ञानिक', '_अनुसंधान', '_में', '_बहुआयामी', 'करण', '_और', '_विद्युतीकरण', '_की', '_महत्वपूर्ण', '_भूमिका', '_है', '।']`	15

8.3 Cross-language comparison

At the **same vocabulary size (50,000)** and the **same selection procedure**, Hindi achieves lower fertility (1.215 vs 1.242), a lower UNK rate (0.253% vs 0.431%), and a lower characters-per-token ratio (4.20 vs 5.20) than Nepali. This is consistent with Hindi's larger, more homogeneous training corpus giving the BPE learner more repeated sub-word patterns to merge into single, longer tokens, while Nepali — despite sharing a script — has a comparatively smaller and more source-concentrated corpus, so more of its text still needs to be represented as shorter, more granular pieces. Both languages nonetheless land at very low UNK rates (<0.5%) at 50k vocabulary, showing character coverage of 0.9995 was sufficient for both.

---

9. Deliverables Mapping

#	Required deliverable	Location in this repo
1	Dataset collection scripts (both languages)	`hindi_web_crawler.py`, `nepali_web_crawler.py` (+ `nepali_web_crawler_v1_69M.py`, the earlier run), `download_and_merge.py`, `download_merge_nepali.py`
2	Dataset preprocessing pipelines	`preprocess_hindi_corpous.py`, `preprocess_nepali_corpus.py`
3	Per-language dataset statistics reports	`dataset_statistics_report.docx`, §5 of this README, `Proof_data_collection_log.log` (raw evidence)
4	Per-language train/ validation/test splits	Produced by `split_train_val_test()` inside each `train_tokenizer_*_final_3_split.py`, written to `tokenizer_work_{lang}/`
5	Tokenizer training code (both languages)	`train_tokenizer_hindi_final_3_split.py`, `train_tokenizer_nepali_final_3_split.py`
6	Vocabulary files (one per language)	`final/hindi_tokenizer.vocab`, `final/nepali_tokenizer.vocab`
7	Tokenizer model files (one per language)	`final/hindi_tokenizer.model`, `final/nepali_tokenizer.model`

---

10. Repository Structure & How to Reproduce

```
LMA_phase1/
├── data_path.txt
├── README.md
├── Hindi/
│   ├── Dataset_collection_script/
│   │   ├── download_and_merge.py
│   │   └── hindi_web_crawler.py
│   ├── Dataset_preprocessing/
│   │   └── preprocess_hindi_corpous.py
│   ├── Tokenizer_model/
│   │   └── hindi_tokenizer.model
│   ├── Tokenizer_training/
│   │   └── train_tokenizer_hindi_final_3_split.py
│   ├── Vocabulary_file/
│   │   └── hindi_tokenizer.vocab
│   └── Results.log.txt
└── Nepali/
    ├── Dataset_collection_script/
    │   ├── download_merge_nepali.py
    │   ├── nepali_web_crawler.py
    │   └── nepali_web_crawler_v1_69M.py
    ├── Dataset_preprocessing/
    │   └── preprocess_nepali_corpus.py
    ├── Tokenizer_model/
    │   └── nepali_tokenizer.model
    ├── Tokenizer_training/
    │   └── train_tokenizer_nepali_final_3_split.py
    ├── Vocabulary_file/
    │   └── nepali_tokenizer.vocab
    └── Results.log.txt
```

To reproduce end-to-end, per language:

```
pip install trafilatura aiohttp aiodns beautifulsoup4 nest_asyncio --break-system-packages
pip install datasets huggingface_hub pyarrow sentencepiece --break-system-packages

# 1. Hindi pipeline
cd Hindi
python Dataset_collection_script/hindi_web_crawler.py
python Dataset_preprocessing/preprocess_hindi_corpous.py
python Dataset_collection_script/download_and_merge.py
python Tokenizer_training/train_tokenizer_hindi_final_3_split.py

# 2. Nepali pipeline
cd ../Nepali
python Dataset_collection_script/nepali_web_crawler_v1_69M.py
python Dataset_collection_script/nepali_web_crawler.py
python Dataset_preprocessing/preprocess_nepali_corpus.py
python Dataset_collection_script/download_merge_nepali.py
python Tokenizer_training/train_tokenizer_nepali_final_3_split.py
```

Run each command from the corresponding `Hindi` or `Nepali` directory. The scripts may contain local absolute paths that need updating when reproducing the pipeline on another machine.

---

11. Limitations, Risks & Future Work

- **Manual-fraction is currently borderline (~19.8–19.9%)**, just under the 20% floor, based on pre-dedup estimated token budgets — see the caveat in §5.3. This needs a final recount on the actual merged, deduplicated corpus using the trained tokenizer (not the 1.7×-word proxy), and possibly one more short crawl top-up.
- **Token counts during crawling/downloading are estimates**, not exact tokenizer output — a fixed 1.7 tokens/word multiplier is used throughout for budgeting. The only exact, tokenizer-derived counts in this report are the TEST-split figures in §8 (3,399,126 tokens for Nepali, 3,376,802 for Hindi), which are on a 1%-of-corpus sample, not the full corpus.
- **Site coverage skew**: both crawls draw heavily from mainstream news portals; literary/long-form sources are present but proportionally small, so register diversity (formal news Hindi/Nepali) likely dominates the manually-collected portion.

- **Devanagari-ratio filtering (line- and document-level)** is a simple heuristic and can both admit some noisy text (e.g. transliterated loanwords) and reject legitimate short lines/quotes under the length or ratio threshold.
- **Public-corpus quality is inherited as-is** from AI4Bharat Sangraha's own "verified" cleaning; this project does not re-clean the public split beyond exact-line deduplication against the crawl.
- **Hardcoded local paths** in every script currently assume a specific Windows machine layout and would need parameterization (e.g. CLI args or a config file) for portability.
- **Future work**: recompute exact final token counts and manual/public split with the trained tokenizers; consider stratified sampling across source categories to balance register; and, for Phase 2, use the tokenizer files in `Hindi/Tokenizer_model/`, `Hindi/Vocabulary_file/`, `Nepali/Tokenizer_model/`, and `Nepali/Vocabulary_file/` as the fixed inputs to model training.

---

12. Appendix: Script Reference

Script	Role
`Hindi/Dataset_collection_script/hindi_web_crawler.py`	Hindi web crawler for manual data collection.
`Hindi/Dataset_collection_script/download_and_merge.py`	Downloads and merges the Hindi public corpus.
`Hindi/Dataset_preprocessing/preprocess_hindi_corpous.py`	Hindi corpus preprocessing pipeline.
`Hindi/Tokenizer_training/train_tokenizer_hindi_final_3_split.py`	Hindi tokenizer training and evaluation.
`Nepali/Dataset_collection_script/nepali_web_crawler_v1_69M.py`	Nepali crawler, first collection run.
`Nepali/Dataset_collection_script/nepali_web_crawler.py`	Nepali crawler, continuation collection run.
`Nepali/Dataset_collection_script/download_merge_nepali.py`	Downloads and merges the Nepali public corpus.
`Nepali/Dataset_preprocessing/preprocess_nepali_corpus.py`	Nepali corpus preprocessing pipeline.
`Nepali/Tokenizer_training/train_tokenizer_nepali_final_3_split.py`	Nepali tokenizer training and evaluation.
`Hindi/Tokenizer_model/hindi_tokenizer.model`	Trained Hindi tokenizer model.
`Hindi/Vocabulary_file/hindi_tokenizer.vocab`	Hindi tokenizer vocabulary.
`Nepali/Tokenizer_model/nepali_tokenizer.model`	Trained Nepali tokenizer model.
`Nepali/Vocabulary_file/nepali_tokenizer.vocab`	Nepali tokenizer vocabulary.
`Hindi/Results.log.txt`, `Nepali/Results.log.txt`	Training and evaluation logs.